Computation of the Invariant Unstable and Stable Manifolds of Periodic Orbits for the RTBP

Ferran Estrella Serra

December 2025

Introduction

In this document, we numerically compute the Lyapunov periodic orbits around the equilibrium point L_3, parametrized by the Jacobi constant, in the planar restricted three-body problem. Then, we introduce an ad hoc method to compute the unstable and stable invariant manifolds associated with these unstable Lyapunov periodic orbits, and we apply it to a specific periodic orbit. Finally, we include the *SageMath* code used for the numerical computations and for generating the plots.

Part 1: Lyapunov Periodic Orbits around Equilibrium Points

This work is a direct continuation of the document *Computation of Symmetric Homoclinic Orbits in the Restricted Three Body Problem*, and we assume that all the material introduced there is known. In particular, we commented that the equilibrium points L_1, L_2, and L_3 are of center–saddle type. In that document, we focused on the saddle part, and in this work we focus instead on the center part, which explains the emergence of unstable periodic orbits (PO) around the points L_i.
In this section, we introduce Lyapunov periodic orbits around the equilibrium points L_i of the RTBP and comment on their stability.

Lyapunov Periodic Orbits

First, let us introduce the following function:

$$C(x_1, x_2, x_3, x_4) = 2\Omega(x_1, x_2) - (x_3^2 + x_4^2) \tag{1}$$

where

$$\Omega(x, y) = \frac{1}{2}(x^2 + y^2) + \frac{1-\mu}{r_1} + \frac{\mu}{r_2} + \frac{1}{2}\mu(1-\mu). \tag{2}$$

The function C is a non-degenerate first integral of the system (this can be checked trivially) and is called the Jacobi first integral.

Since C is a non-degenerate first integral and L_i is a center–saddle equilibrium for $i = 1, 2, 3$, the *Lyapunov theorem* on the emergence of periodic orbits guarantees the **existence of a one-parameter family of periodic orbits emanating from the equilibrium point** L_i. We call these orbits **Lyapunov periodic orbits (LPO).**

The Monodromy Matrix

Second, let us denote by $\phi(t, \mathbf{x})$, with $\mathbf{x} = (x_1, x_2, x_3, x_4)$, the (local) flow associated with the RTBP system $\dot{\mathbf{x}} = F(\mathbf{x})$, and by $D\phi(t, \mathbf{x})$ the principal fundamental matrix of the first-order variational equation (with respect to $\mathbf{x}$):

$$\dot{\mathbf{y}} = DF(\phi(t, \mathbf{x}))\,\mathbf{y}. \tag{3}$$

If we assume that $\mathbf{x}$ is a T-periodic point (for instance, a point on a LPO), that is,

$$\phi(t + T, \mathbf{x}) = \phi(t, \mathbf{x}) \tag{4}$$

for all t, then the first-order variational equation is a T-periodic homogeneous linear system, since

$$DF(\phi(t+T,\mathbf{x})) = DF(\phi(t,\mathbf{x})) \tag{5}$$

for all t. Consequently, the monodromy matrix (from Floquet theory) associated with the first-order variational equation is

$$D\phi(T,\mathbf{x}). \tag{6}$$

The eigenvalues of the monodromy matrix, called the characteristic multipliers, coincide for the monodromy matrices $D\phi(T,\mathbf{x})$ evaluated at different T-periodic points $\mathbf{x}$ along the same periodic orbit. More precisely, if $\mathbf{x}$ is a T-periodic point and the monodromy matrix $D\phi(T,\mathbf{x})$ has an eigenvector v_0 with eigenvalue α, then

$$v(t) = \alpha^{-t/T} D\phi(t,\mathbf{x})\, v_0 \tag{7}$$

is a T-periodic parametrization of an eigenvector $v = v(t)$ associated with the eigenvalue α of the monodromy matrix $D\phi(T,\phi(t,\mathbf{x}))$.

For the RTBP, since C is a non-degenerate first integral of the system, the monodromy matrices have 1 as a characteristic multiplier with multiplicity greater than one. In fact, for the RTBP, the characteristic multipliers are of the form $1, 1, \lambda$, and $1/\lambda$.

This implies that, for periodic orbits whose characteristic multiplier λ is different from 1, the orbit is an elementary periodic orbit. By the *Cylinder Theorem*, such periodic orbits lie on a cylinder (that is, a family) of periodic orbits parametrized by the value of the first integral.
Therefore, for elementary LPOs, the family of periodic orbits can be parametrized by the value of the Jacobi first integral C.

Stability of Periodic Orbits

We now study the stability of the periodic orbits. Given a periodic point $\mathbf{x}$ on a T-periodic orbit, the stability of the orbit can be studied by analysing the stability of $\mathbf{x}$ as a fixed point of the T-flow map

$$\mathbf{x} \mapsto \phi(T,\mathbf{x}). \tag{8}$$

That is, we analyse the eigenvalues of the differential of the T-flow map at a periodic point, which are the characteristic multipliers of the monodromy matrix at that point. We recall that these characteristic multipliers are independent of the chosen point along the periodic orbit.

For the RTBP, we have commented that the characteristic polynomial of the monodromy matrix can be expressed as

$$(s-1)^2(s-\lambda)(s-1/\lambda) = (s-1)^2(s^2-ks+1) \tag{9}$$

where

$$k = \lambda + 1/\lambda \tag{10}$$

is known as the stability parameter. In particular, we have that

$$k = \mathrm{tr}\big(D\phi(T,\mathbf{x})\big) - 2. \tag{11}$$

We may study the stability of the periodic orbit in terms of the stability parameter k:

- If $k \in (-2,2)$, then λ and $1/\lambda$ are complex conjugates lying on the unit circle. We say that the periodic orbit is linearly stable.
- If $k \in \{-2,2\}$, then $\lambda, 1/\lambda = 1, -1$. We say that the periodic orbit is critical.
- If $k \in \mathbb{R}\backslash[-2,2]$, then $|\lambda| > 1$ and $|1/\lambda| < 1$. We say that the periodic orbit is unstable. In this case, there exist one-dimensional unstable and stable invariant manifolds of the fixed point for the T-flow map, and consequently unstable and stable invariant manifolds of the periodic orbit for the flow.

From this analysis, we deduce that **for LPOs with stability parameter $k \in \mathbb{R} \setminus [-2,2]$, the LPOs are elementary and unstable, and have associated unstable and stable invariant manifolds, which we denote, respectively, by**

$$W^u(\mathrm{LPO}), \quad W^s(\mathrm{LPO}). \tag{12}$$

Part 2: Computation of the Lyapunov Periodic Orbits

In this section, we explain the ad hoc method used to compute the Lyapunov periodic orbits around the equilibrium points of the RTBP system.

Let us focus on the periodic orbits around the equilibrium point L_3. A method to find a Lyapunov periodic orbit with Jacobi constant C around L_3 is as follows:

- Take an initial condition on the x-axis with $x > x_{L_3}$, $x' = 0$, and Jacobi constant C. In this case, the value of y' is determined by relation (1):

$$y' = \pm\sqrt{2\Omega(x,0) - C}, \tag{13}$$

  and we choose the negative sign for y'.

- Integrate the system forward in time until the orbit crosses the x-axis. If at the crossing the orbit has $x' = 0$, then by the symmetry of the system $(x, y, x', y', t) \mapsto (x, -y, -x', y')$, the orbit obtained by integration for $y \leq 0$ and its symmetric counterpart for $y \geq 0$ form a periodic orbit around L_3 with Jacobi constant C.

- For the fixed C, to find the initial x such that $x' = 0$ at crossing, proceed by computing x' at crossing for different initial x until a change of sign is detected. Then, compute the precise x such that $x' = 0$, for instance, by bisection.

The explicit algorithm is included in the *SageMath* code. It is important to note that not all values of C will yield a Lyapunov periodic orbit. For instance, in (13) we must have x and C such that $2\Omega(x, 0) \geq C$, and more in general, the initial conditions must lie outside the forbidden regions of the RTBP.

The following plots are obtained using the previous algorithm, for $\mu = 0.1$ (we will use this mass parameter for all computations and plots from now on).

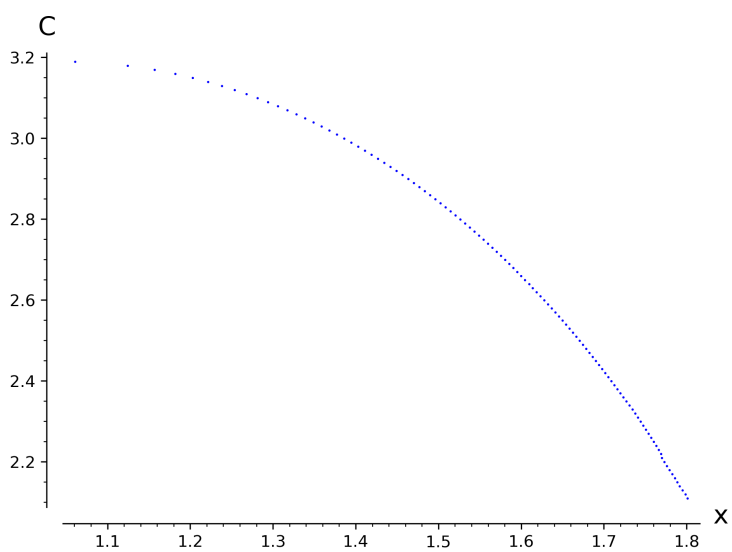

Figure 1: Plot of the Jacobi constant C with respect to the first component of the initial conditions along the x-axis to the right of L_3, for the LPOs around L_3.

In Figure 1, we note that there exists an upper bound for the C values, which is the Jacobi constant at L_3:

$$C_{\max} = C(L_3) = 3.189578150449. \tag{14}$$

There also exists a lower bound, $C_{\min} \approx 2.1$, corresponding to the case in which the orbit obtained by integration collides with the first primary.

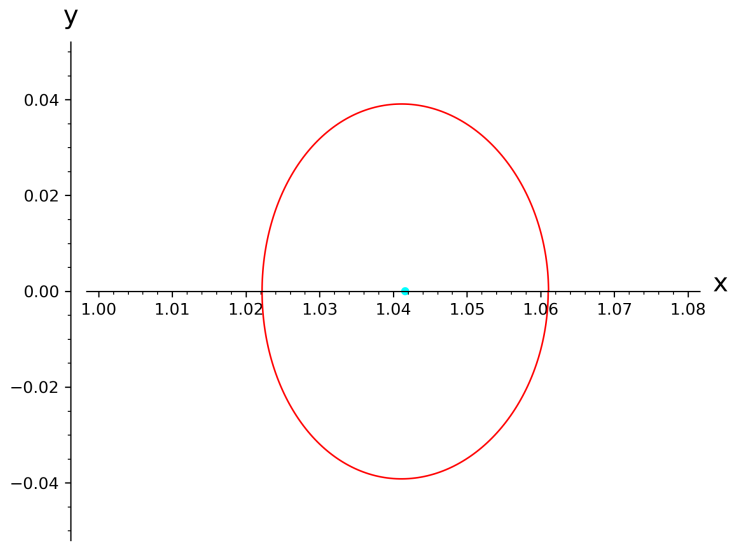

Figure 2: Plot of the Lyapunov periodic orbit around L_3 (cyan dot), obtained for $C = 3.189$ and $x = 1.061020106306$.

Part 3: Computation of the Unstable and Stable Invariant Manifolds of a Lyapunov Periodic Orbit

In this section, we compute the unstable invariant manifold associated with a Lyapunov periodic orbit around L_3. We note that once the unstable invariant manifold is computed, there is no need to compute the stable manifold, since it is its symmetric counterpart due to the symmetry of the system.

Computation of the Unstable Invariant Manifold of a LPO

We compute the unstable invariant manifold associated with an elementary LPO.

As an example, we consider the LPO shown in Figure 2, which has initial first coordinate $x = 1.061020106306$ and corresponds to the Jacobi constant $C = 3.189$. If we compute k in this case, we obtain a $k > 2$, thus the characteristic multiplier λ is larger than 1, and we can compute it numerically.

Let us now describe the method we used to compute the unstable invariant manifold of the considered LPO, which we denote by $W^u = W^u(\text{LPO})$:

- Let $\mathbf{x}$ be a T-periodic point (for instance, the one on the x-axis), and let v_0 be the unit eigenvector associated with the eigenvalue $\lambda > 1$ of the monodromy matrix $D\phi(T, \mathbf{x})$. We choose v_0 such that $v_0^1 \geq 0$.
- Let $v(\tau) = \lambda^{-\tau/T} D\phi(\tau, \mathbf{x})\, v_0$ be the eigenvector of eigenvalue λ of the monodromy matrix $D\phi(T, \phi(\tau, \mathbf{x}))$ for $\tau \in [0, T)$, according to expression (7). Let $\hat{v}(\tau)$ be its normalization.
- Consider the set of initial conditions

  $$\{\mathbf{x}_\pm(\tau) = \phi(\tau, \mathbf{x}) \pm s \cdot \hat{v}(\tau) : \tau \in [0, T),\ \ s > 0 \text{ small}\}.$$

  Numerically, one may take a partition of $[0, T)$, say $\tau_1, \ldots, \tau_k \in [0, T)$, and the corresponding $2k$ initial conditions $\mathbf{x}_\pm(\tau_1), \ldots, \mathbf{x}_\pm(\tau_k)$.
- For each initial condition $\mathbf{x}_\pm$, integrate $\phi(t, \mathbf{x}_\pm)$ forward in time ($t \geq 0$).

  Then, we obtain the positive and negative branches of W^u:

  $$W^{u,+} = \bigsqcup_{\tau \in [0,T)} \{\phi(t, \mathbf{x}_+(\tau)) : t \geq 0\},$$

  $$W^{u,-} = \bigsqcup_{\tau \in [0,T)} \{\phi(t, \mathbf{x}_-(\tau)) : t \geq 0\}.$$

  Hence, the unstable manifold associated with the LPO is

  $$W^u = W^{u,+} \sqcup W^{u,-} \sqcup \{\phi(\tau, \mathbf{x}) : \tau \in [0, T)\}.$$

The explicit formulation of the previous method is included in the *SageMath* code.

Let us now comment on the plots we obtained for the unstable manifold of the considered LPO. In all the plots, the magenta dots indicate the positions of the primaries, P_1 and P_2, and the cyan dots indicate the equilibrium points of the system, L_i for $i = 1, 2, 3, 4, 5$. Also, all plots are obtained using $k = 50$ orbits.

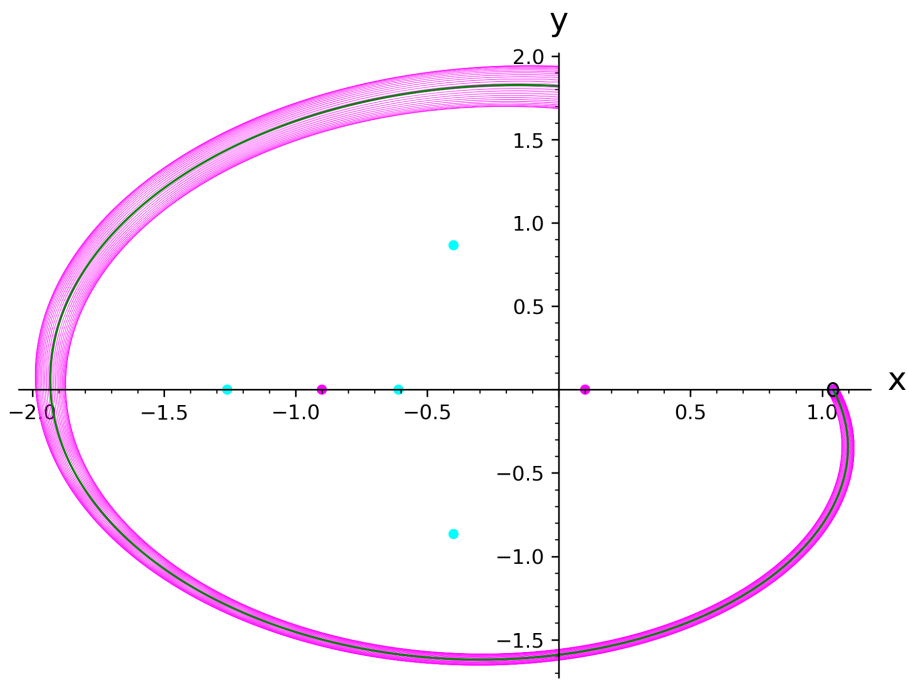

Figure 3: Plot of the (x, y)-projection of the branch $W^{u,+}$ (in purple) associated with the considered LPO (in black), up to the second crossing with $x = 0$. The green curve represents the (x, y)-projection of the positive branch of the unstable invariant curve associated with L_3.

We note that the branch is two-dimensional, tubular around the invariant green curve, and widens as time progresses.

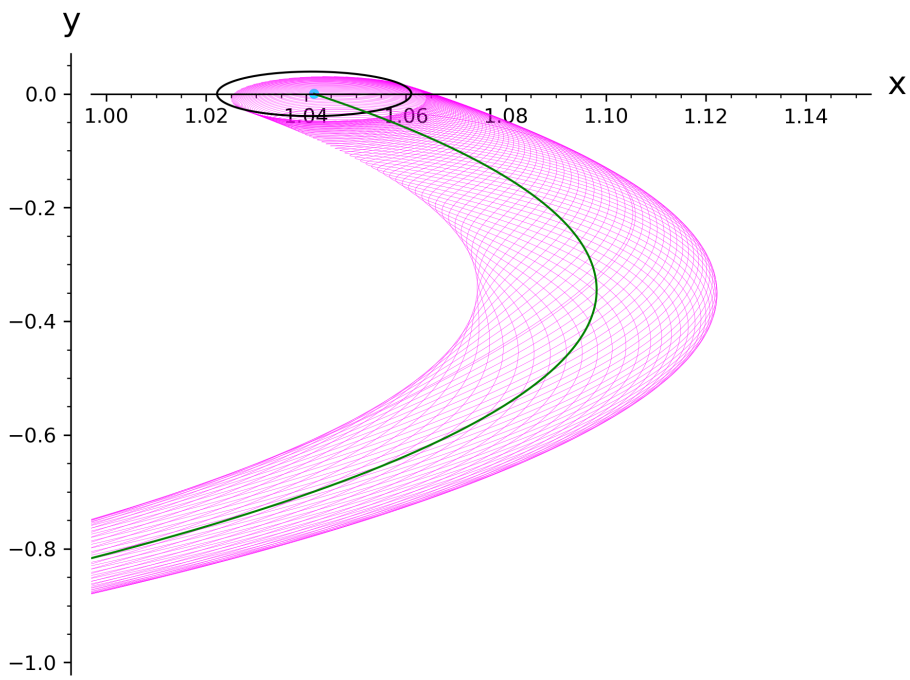

Figure 4: Zoom of the previously discussed plot. We observe the individual $k = 50$ orbits used to construct the unstable manifold, which spiral around the green invariant curve.

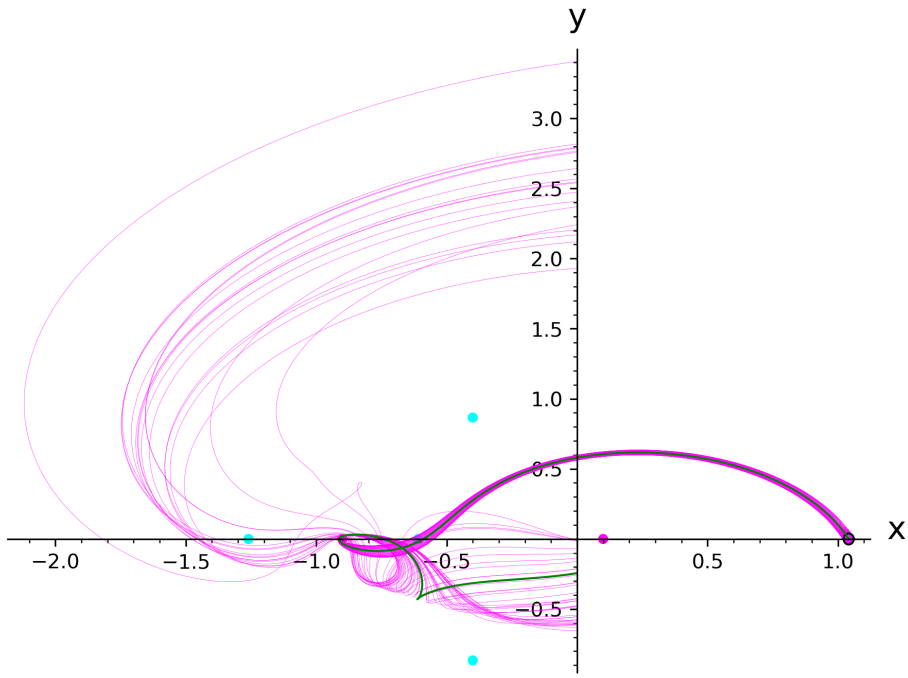

Figure 5: Plot of the (x, y)-projection of the branch $W^{u,-}$ (in purple) associated with the considered LPO (in black), up to the second crossing with $x = 0$. The green curve represents the (x, y)-projection of the negative branch of the unstable invariant curve associated with L_3.

We note that the branch is two-dimensional and tubular around the invariant green curve until it approaches the second primary. Beyond this point, the orbits become chaotically dispersed, representing either a collision with or a close approach to the second primary.

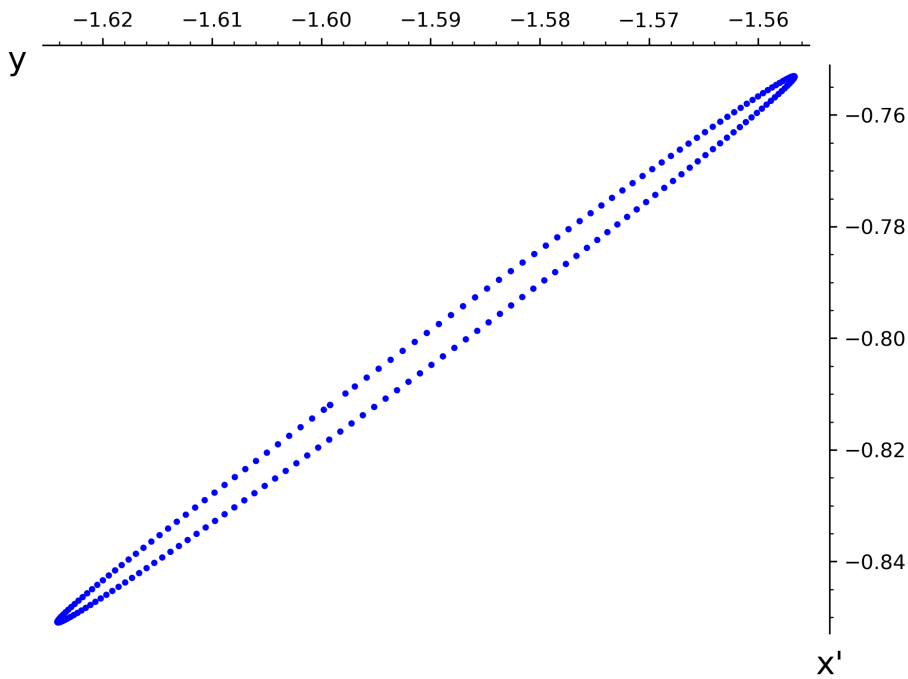

Figure 6: Plot of the (y, x')-projection of the intersection between $x = 0$ and $W^{u,+}$.

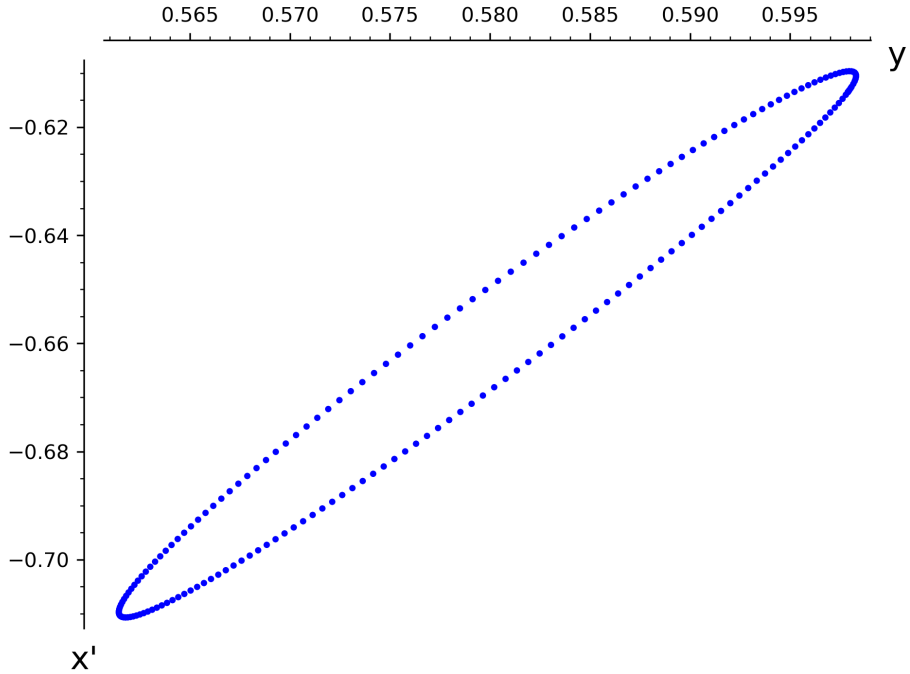

Figure 7: Plot of the (y, x')-projection of the intersection between $x = 0$ and $W^{u,-}$.

Appendix: *SageMath* Code

```
#! /usr/bin/env sage
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import csv

from matplotlib.colors import Normalize
from scipy.integrate import solve_ivp
from mpl_toolkits.mplot3d import Axes3D

#OPTION 1: Data file with the periodic points at y=0 with
    the corresponding C
#OPTION 2: Read data file and generate new data file with
    the periodic points at y=0 with the corresponding k at
    intersection
#OPTION 3: Read data files x0vsk and x0vsC and plot them
#OPTION 4: PO plot
#OPTION 5: Plot of the invariant curve associated to L3.
    Positive and negative branch
#OPTION 6: Plot the unstable curve of just one periodic
    point in the LPO
#OPTION 7: Plot the unstable manifold of the LPO

option=
plot=

#SYSTEM AND INTEGRATION:

def g(t,x):
    return x[1]

def f(t,P,mu):
    x1=P[0]
    x2=P[1]
    x3=P[2]
    x4=P[3]
    o1=omegax(x1,x2,mu)
    o2=omegay(x1,x2,mu)
    return [x3,x4,2*x4+o1,-2*x3+o2]

def negf(t,P,mu):
    return [-x for x in f(t,P,mu)]

#F=f+variational equations:

def Df(P,mu):
```

```
    x1=P[0]
    x2=P[1]
    oxx=omegaxx(x1,x2,mu)
    oxy=omegaxy(x1,x2,mu)
    oyy=omegayy(x1,x2,mu)
    return [[0,0,1,0],[0,0,0,1],[oxx,oxy,0,2],[oxy,oyy
        ,-2,0]]

def X(P):
    return [
    [P[4], P[5], P[6], P[7]],
    [P[8], P[9], P[10], P[11]],
    [P[12], P[13], P[14], P[15]],
    [P[16], P[17], P[18], P[19]]]

def F(t,P,mu):
    x1=P[0]
    x2=P[1]
    x3=P[2]
    x4=P[3]
    o1=omegax(x1,x2,mu)
    o2=omegay(x1,x2,mu)
    ret=[x3,x4,2*x4+o1,-2*x3+o2]

    A=Df(P,mu)
    B=X(P)
    for i in range(4):
        s=[]
        for j in range(4):
            d=0
            for k in range(4):
                d+=A[i][k]*B[k][j]
            s.append(d)
        ret+=s

    return ret


def negF(t,P,mu):
    return [-x for x in F(t,P,mu)]


def L1(mu,TOL):
    def g(x,mu):
        a=mu*(1-x)**2
        b=3-2*mu-x*(3-mu-x)
        return (a/b)**(1/3)
    x0=(mu/(3-3*mu))**1/3
    while True:  #We do not write a break option because
        theoretically we know the numerical computation will
```

```
        work.
        x1=g(x0,mu)
        if abs(x1-x0)<TOL:
            return [mu-1+x1,0,0,0]
        x0=x1

def L2(mu,TOL):
    def f(x,mu):
        a=mu*(1+x)**2
        b=3-2*mu+x*(3-mu+x)
        return (a/b)**(1/3)
    x0=(mu/(3-3*mu))**1/3
    while True:
        x1=f(x0,mu)
        if abs(x1-x0)<TOL:
            return [mu-1-x1,0,0,0]
        x0=x1

def L3(mu,TOL):
    def l(x,mu):
        a=(1-mu)*(1+x)**2
        b=1+2*mu+x*(2+mu+x)
        return (a/b)**(1/3)
    x0=1-7*mu/12
    while True:
        x1=l(x0,mu)
        if abs(x1-x0)<TOL:
            return [mu+x1,0,0,0]
        x0=x1

def L4(mu):
    return [mu-0.5,3**0.5/2,0,0]

def L5(mu):
    return [mu-0.5,-3**0.5/2,0,0]

def P1(mu):
    return [mu,0]

def P2(mu):
    return [mu-1,0]




#FUNCTIONS AND JACOBI 1st INTERGAL

def r1(x,y,mu):
    a=(x-mu)**2+y**2
    return sqrt(a)
```

```
def r2(x,y,mu):
    a=(x-mu+1)**2+y**2
    return sqrt(a)

def omega(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return (x**2+y**2)/2+(1-mu)/rr1+mu/rr2+mu*(1-mu)/2

def omegax(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return x-(1-mu)*(x-mu)/rr1**3-mu*(x-mu+1)/rr2**3

def omegay(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return y*(1-(1-mu)/rr1**3-mu/rr2**3)

def omegaxx(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return 1-(1-mu)/rr1**3+3*(1-mu)*(x-mu)**2/rr1**5-mu/rr2
        **3+3*mu*(x-mu+1)**2/rr2**5

def omegaxy(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return y*(3*(1-mu)*(x-mu)/rr1**5+3*mu*(x-mu+1)/rr2**5)

def omegayy(x,y,mu):
    rr1=r1(x,y,mu)
    rr2=r2(x,y,mu)
    return 1-(1-mu)/rr1**3-mu/rr2**3+y**2*(3*(1-mu)/rr1
        **5+3*mu/rr2**5)

def Cfunc(P,mu):
    x1=P[0]
    x2=P[1]
    x3=P[2]
    x4=P[3]
    o=omega(x1,x2,mu)
    return 2*o-(x3**2+x4**2)


#STABILITY:

def charexp(P,mu,TOL):
    x1=P[0]
```

```
x2=P[1]
x3=P[2]
x4=P[3]
oxx=omegaxx(x1,x2,mu)
oxy=omegaxy(x1,x2,mu)
oyy=omegayy(x1,x2,mu)
A=np.array([[0,0,1,0],[0,0,0,1],[oxx,oxy,0,2],[oxy,oyy
    ,-2,0]])
evals,evecs=np.linalg.eig(A)

cat1,cat2,cat3,cat4,cat5,cat6,cat7,cat8
    =[],[],[],[],[],[],[],[]
for ev in evals:
    re=ev.real
    im=ev.imag
    if abs(im)<TOL:
        if re>TOL:
            cat1.append(ev)
        elif re<-TOL:
            cat2.append(ev)
    elif re>TOL:
        if im>TOL:
            cat3.append(ev)
        elif im<-TOL:
            cat4.append(ev)
    elif re<-TOL:
        if im>TOL:
            cat5.append(ev)
        elif im<-TOL:
            cat6.append(ev)
    elif abs(re)<TOL:
        if im>TOL:
            cat7.append(ev)
        elif im<-TOL:
            cat8.append(ev)
evals_sorted=np.array(cat1+cat2+cat3+cat4+cat5+cat6+cat7
    +cat8)


evecs_sorted=[]
for a in evals_sorted:
    i=np.where(np.isclose(evals,a))[0][0]
    v=evecs[:,i]

    v=v/np.linalg.norm(v)
    if v[0].real<0:
        v=-v

    evecs_sorted.append(v)
evals=[]
```

```
    for a in evals_sorted:
        if abs(a.imag)<TOL:
            evals.append(a.real)
        elif abs(a.real)<TOL:
            evals.append(a.imag*1j)
        else:
            evals.append(a)

    evals=np.array(evals)

    evecs=[]
    for v in evecs_sorted:
        v_aux=[]
        for a in v:
            if abs(a.imag)<TOL:
                v_aux.append(a.real)
            elif abs(a.real)<TOL:
                v_aux.append(a.imag)
            else:
                v_aux.append(a)
        evecs.append(v_aux)

    evecs=np.array(evecs)

    return A,evals,evecs




#INVARIANT MANIFOLDS ASSOCIATED TO THE Li

def h(t,x):
    return x[0]

#Type 0=Unstable manifold
#Type 1=Stable manifold
#iregion 1=Positive branch
#iregion -1=Negative branch

def invariant_manifold_Li(type,i,iregion,tf,ncrossing,mu,TOL
    ):

    if i==1: P=L1(mu,TOL)
    elif i==2: P=L2(mu,TOL)
    elif i==3: P=L3(mu,TOL)
    else: return False

    v_standard=charexp(P,mu,TOL)[2][type]
    v=[]
```

```
    for a in v_standard:
        if iregion==1:
            v.append(a)
        elif iregion==-1:
            v.append(-a)
        else: return False

    v=np.array(v)
    s=1e-6

    x0=[P[0]+s*v[0],P[1]+s*v[1],P[2]+s*v[2],P[3]+s*v[3]]



    T=0
    points=[]

    for j in range(0,ncrossing):
        if type==0: #Forward in time
            f_fix=lambda t,x: f(t,x,mu)
            h.terminal=True
            solution=solve_ivp(f_fix,(T,T+tf),x0,atol=TOL,
                max_step=0.01,events=h)

        if type==1: #Backward in time, assuming tf>0
            f_fix=lambda t,x: negf(t,x,mu)
            h.terminal=True
            solution=solve_ivp(f_fix,(T,T+tf),x0,atol=TOL,
                max_step=0.01,events=h)

        T=solution.t_events[0][0]
        x0_event=np.array(solution.y_events[0][0])
        f0=np.array(f_fix(T, x0_event))
        x0=x0_event+1e-10*f0
        points.append(solution.y)


    return points





##########################################################################################
```

```
#1) Finding the LPOs (adhoc method)

#1.1) Initial condition at y=0 depending on C
def IC_xaxis(x,C,mu):
    x1=x
    x2=-TOL*1.1   #y component for an IC in the x axis at
        the right of L3 is taken to be close to zero and
        negative (according to the negative sign
                # of the y comp of the velocity). This is a
                    required step for the integration to
                    proceed (abs(y)>TOL)
    x3=0.
    #x4:

    o=omega(x,0.,mu)
    r=2*o-C
    if r<0:
        raise ValueError("Invalid C on considered (x,0,0,y')
            ")
    else:
        x4=-sqrt(r)         #y component of the velocity for
             an IC in the x axis at the right of L3 is taken
            to be negative (a priori info)

    return [x1,x2,x3,x4]





#1.2) Computation of the orbit starting at y=0 and ending at
     first crossing with y=0. Will define the function F(x)=x
    ,
def orbit_crossing(x,C,mu,tf,timedirection,TOL):
    x0=IC_xaxis(x,C,mu)

    if timedirection>0: #Forward in time
        f_fix=lambda t,x: f(t,x,mu)
        g.terminal=True
        solution=solve_ivp(f_fix,(0,tf),x0,atol=TOL,events=g
            )

    if timedirection<0: #Backward in time, assuming tf>0
        f_fix=lambda t,x: negf(t,x,mu)
        g.terminal=True
        solution=solve_ivp(f_fix,(0,tf),x0,atol=TOL,events=g
            )
```

```
    crossingpoint=solution.y_events[0][0]

    return crossingpoint[2]




#1.3) Method to find F(x)=0: localization and bisection.
def zerofinding(F,xini,xincrement,TOL):
    if F(xini)*F(xini+xincrement)<0:
        return bisection(F,xini,xini+xincrement,TOL)
    else:
        return zerofinding(F,xini+xincrement,xincrement,TOL)

def bisection(F,a,b,TOL): #We assume F continuous, therefore
     the method will converge
    Fa=F(a)
    Fb=F(b)
    if a>b or Fa*Fb>0:
        raise ValueError("Invalid interval")
    if abs(Fa)<TOL:
        return a
    if abs(Fb)<TOL:
        return b

    m=(a+b)/2
    Fm=F(m)

    if Fa*Fm<0:
        return bisection(F,a,m,TOL)
    else:
        return bisection(F,m,b,TOL)




#1.4) Once LPO localized, let us find the period and the
    points of the orbit

def periodic_orbit(x,C,mu,tf,timedirection,TOL):
    x0=IC_xaxis(x,C,mu)+[1 if i%5==0 else 0 for i in range
        (5,21)]
    #Find period
    T=0
    xf=x0
    for j in range(0,2):
        if timedirection>0: #Forward in time
            F_fix=lambda t,x: F(t,x,mu)
            g.terminal=True
```

```
            solution=solve_ivp(F_fix,(T,T+tf),xf,atol=TOL,
                max_step=0.01,events=g)

        if timedirection<0: #Backward in time, assuming tf>0
            F_fix=lambda t,x: negF(t,x,mu)
            g.terminal=True
            solution=solve_ivp(F_fix,(T,T+tf),xf,atol=TOL,
                max_step=0.01,events=g)

        T=solution.t_events[0][0]
        xf=solution.y_events[0][0]

    #Find periodic orbit
    if timedirection>0: #Forward in time
        F_fix=lambda t,x: F(t,x,mu)
        solution=solve_ivp(F_fix,(0,T),x0,atol=TOL,max_step
            =0.01)

    if timedirection<0: #Backward in time, assuming tf>0
        F_fix=lambda t,x: negF(t,x,mu)
        solution=solve_ivp(F_fix,(0,T),x0,atol=TOL,max_step
            =0.01)

    points=solution.y
    xf=solution.y[:, -1]

    return T,points


#1.5) Once the T-LPO computed, let us find the monodromy
    matrix and the k associated to a periodic point x0 and
    the eigenvectors and eigenvalues


def monodromy(x0,T,mu):                    #Assume x0 in the
     LPO of period T
    x0=x0+[1 if i%5==0 else 0 for i in range(5,21)]
    if timedirection>0: #Forward in time
        F_fix=lambda t,x: F(t,x,mu)
        solution=solve_ivp(F_fix,(0,T),x0,atol=TOL,max_step
            =0.01)

    if timedirection<0: #Backward in time, assuming tf>0
        F_fix=lambda t,x: negF(t,x,mu)
        solution=solve_ivp(F_fix,(0,T),x0,atol=TOL,max_step
            =0.01)

    xf=solution.y[:, -1]

    M = [
```

```
    [xf[4],  xf[5],  xf[6],  xf[7]],
    [xf[8],  xf[9],  xf[10], xf[11]],
    [xf[12], xf[13], xf[14], xf[15]],
    [xf[16], xf[17], xf[18], xf[19]],
    ]


    k=xf[4]+xf[9]+xf[14]+xf[19]-2

    return k,M


def charmult(x0,T,mu,TOL):

    M=monodromy(x0,T,mu)[1]
    evals,evecs=np.linalg.eig(M)

    cat1,cat2,cat3,cat4,cat5,cat6,cat7,cat8
        =[],[],[],[],[],[],[],[]
    for ev in evals:
        re=ev.real
        im=ev.imag
        if abs(im)<TOL:
            if re>TOL:
                cat1.append(ev)
            elif re<-TOL:
                cat2.append(ev)
        elif re>TOL:
            if im>TOL:
                cat3.append(ev)
            elif im<-TOL:
                cat4.append(ev)
        elif re<-TOL:
            if im>TOL:
                cat5.append(ev)
            elif im<-TOL:
                cat6.append(ev)
        elif abs(re)<TOL:
            if im>TOL:
                cat7.append(ev)
            elif im<-TOL:
                cat8.append(ev)
    evals_sorted=np.array(cat1+cat2+cat3+cat4+cat5+cat6+cat7
        +cat8)


    evecs_sorted=[]
    for a in evals_sorted:
        i=np.where(np.isclose(evals,a))[0][0]
        v=evecs[:,i]
```

```
        v=v/np.linalg.norm(v)
        if v[0].real<0:
            v=-v

        evecs_sorted.append(v)
    evals=[]
    for a in evals_sorted:
        if abs(a.imag)<TOL:
            evals.append(a.real)
        elif abs(a.real)<TOL:
            evals.append(a.imag*1j)
        else:
            evals.append(a)

    evals=np.array(evals)

    evecs=[]
    for v in evecs_sorted:
        v_aux=[]
        for a in v:
            if abs(a.imag)<TOL:
                v_aux.append(a.real)
            elif abs(a.real)<TOL:
                v_aux.append(a.imag)
            else:
                v_aux.append(a)
        evecs.append(v_aux)

    evecs=np.array(evecs)

    return evals,evecs




#1.6) Find the unstable manifold associated to a point of
    the T-LPO

def unstable_manifold_point(iregion,x0,T,tf,ncrossing,mu,TOL
    ):

    x=x0
    v=charmult(x0,T,mu,TOL)[1][0]
    s=1e-6
    for i in range(4):
        x[i]+=iregion*s*v[i].real


    T=0
```

```
    points=[]

    for j in range(0,ncrossing):
        f_fix=lambda t,x: f(t,x,mu)
        h.terminal=True
        solution=solve_ivp(f_fix,(T,T+tf),x,atol=TOL,
            max_step=0.01,events=h)

        T=solution.t_events[0][0]
        x_event=np.array(solution.y_events[0][0])
        f0=np.array(f_fix(T, x_event))
        x=x_event+1e-10*f0
        points.append(solution.y)

    return points




#1.7) Find the unstable manifold associated to the T-LPO

def unstable_manifold_LPO(iregion,x,C,tf,ncrossing,norbits,
    mu,TOL):

    T,sol=periodic_orbit(x,C,mu,tf,timedirection,TOL)
    PP=sol.T
    #Computation unstable eigenvector of the monodromy
    x0=IC_xaxis(x,C,mu)
    v=charmult(x0,T,mu,TOL)[1][0]
    for i in range(4):
        v[i]=v[i].real

    print("v:")
    print(v)
    #Integration on each periodic point
    n=len(PP)//norbits
    k=0
    points_total=[]

    while k<len(PP):

        x=PP[k]
        N=[
        [x[4],  x[5],  x[6],  x[7]],
        [x[8],  x[9],  x[10], x[11]],
        [x[12], x[13], x[14], x[15]],
        [x[16], x[17], x[18], x[19]],
        ]

        x=[x[0],x[1],x[2],x[3]]
```

```
        print("x0:")
        print(x)
        #Computation unstable eigenvector of the monodromy
            at each PP (transport)
        w=[0]*4
        for i in range(4):
            for j in range(4):
                w[i]+=N[i][j]*v[j]

        w=w/np.linalg.norm(w)
        print("w")
        print(w)


        s=1e-2
        for i in range(4):
            x[i]+=iregion*s*w[i]

        print("x")
        print(x)
        T=0
        points=[]




        for j in range(0,ncrossing):
            f_fix=lambda t,x: f(t,x,mu)
            h.terminal=True
            solution=solve_ivp(f_fix,(T,T+tf),x,atol=TOL,
                max_step=0.01,events=h)

            T=solution.t_events[0][0]
            x_event=np.array(solution.y_events[0][0])
            f0=np.array(f_fix(T, x_event))
            x=x_event+1e-10*f0
            points.append(solution.y)

        points_total.append(points)
        k+=n

    return points_total


##########################################################################################



#MAIN:
```

```
mu=0.1
TOL=1e-12


#Setup for Lyapunov PO around L3

l3=L3(mu,TOL)
Cl3=Cfunc(l3,mu)
print(Cl3)

#We choose a C range... (must have a priori knowledge about
    the system: related with C(L3))

Cmin=2.1
Cmax=3.189
Cincrement=1e-2


#We choose for the integration... (valid for all the C's)
xini=1.041609   #We are on the RIGHT (larger x, positive) of
     L3. This is key and necessary to know (a priori info
    about the system)
xincrement=1e-2
tf=20
timedirection=+1  #for time advancing the orbit of the
    considered IC will go to negative y (a priori info)


#OPTION 1: Data file with the periodic points at y=0 with
    the corresponding C
if option==1:
    with open("data11_x0vsC.csv", "a", newline="") as file:
        writer=csv.writer(file)

        C=Cmax
        while C>=Cmin:
            F=lambda x: orbit_crossing(x,C,mu,tf,
                timedirection,TOL)
            xzero=zerofinding(F,xini,xincrement,TiOL)

            writer.writerow([xzero,C])
            xini=xzero   #IMPORTANT: Extra step, which uses
                that a priori we know that as C decreases the
                 IC x st we obtain the PO increases.
            C-=Cincrement


#OPTION 2: Read data file and generate new data file with
    the periodic points at y=0 with the corresponding k at
    intersection
```

```
if option==2:
    with open("data11_x0vsk.csv", "a", newline="") as file:
        writer=csv.writer(file)

        df=pd.read_csv("data11_x0vsC.csv", header=None)
        xzero_vals  = df.iloc[:,int(0)].tolist()
        C_vals  = df.iloc[:,int(1)].tolist()

        pairs=list(zip(xzero_vals, C_vals))

        for p in pairs:
            x0=IC_xaxis(p[0],p[1],mu)
            T=periodic_orbit(p[0],p[1],mu,tf,timedirection,
                TOL)[0]
            k=monodromy(x0,T,mu)[0]
            writer.writerow([p[0],k])
            print([p[0],k])


#OPTION 3: Read data files x0vsk and x0vsC and plot them
if option==3:

    #Plot k(x)
    df=pd.read_csv("data11_x0vsk.csv", header=None)
    xzero_vals  = df.iloc[:,int(0)].tolist()
    k_vals  = df.iloc[:,int(1)].tolist()


    plots=[]
    plots.append(list_plot(list(zip(xzero_vals, k_vals)),
        size=2))

    plot_total=sum(plots)
    plot_total.axes_labels(['x','k'])
    plot_total.show(gridlines=True)
    plot_total.save(f'A11_xvsk.png',dpi=300)


    #Plot C(x)
    df=pd.read_csv("data11_x0vsC.csv", header=None)
    xzero_vals  = df.iloc[:,int(0)].tolist()
    C_vals  = df.iloc[:,int(1)].tolist()


    plots=[]
    plots.append(list_plot(list(zip(xzero_vals,k_vals)),size
        =2))
    plots.append(line([(min(xzero_vals),2),(max(xzero_vals)
        ,2)],linestyle='--',thickness=1,color='black'))
    plot_total=sum(plots)
```

```
    plot_total.axes_labels(['x','k'])
    plot_total.show(gridlines=True)
    plot_total.save('A11_xvsk.png',dpi=300)


#OPTION 4: PO plot
if option==4:
    C=3.189
    x=1.06102010630636
    plots=[]

    x_list_u=periodic_orbit(x,C,mu,tf,timedirection,TOL)[1]
    plots.append(list_plot(list(zip(x_list_u[0],x_list_u[1])
        ),color='red',plotjoined=True))

    primaries=[P1(mu),P2(mu)]
    plots.append(list_plot(primaries, color='magenta', size
        =25))

    equilibrium=[[L1(mu,TOL)[0],L1(mu,TOL)[1]],[L2(mu,TOL)
        [0],L2(mu,TOL)[1]],[L3(mu,TOL)[0],L3(mu,TOL)[1]],[L4(
        mu)[0],L4(mu)[1]],[L5(mu)[0],L5(mu)[1]]]
    plots.append(list_plot(equilibrium, color='cyan', size
        =25))



    plot_total=sum(plots)
    plot_total.set_axes_range(1, 1.08, -0.05, 0.05)
    plot_total.axes_labels(['x','y'])
    plot_total.show(gridlines=True)
    plot_total.save(f'A11-LPO.png',dpi=300)



#OPTION 5: Plot of the invariant curve associated to L3.
    Positive and negative branch
if option==5:
    type=0 #Unstable
    i=3 #L3
    iregion_values=[-1,1]

    for iregion in iregion_values:
        plots=[]
        points=invariant_manifold_Li(type,i,iregion,100,2,mu
            ,TOL)
        for p in points:
            x_list_u=p
            plots.append(list_plot(list(zip(x_list_u[0],
                x_list_u[1])),color='green',plotjoined=True))
```

```
        primaries=[P1(mu),P2(mu)]
        plots.append(list_plot(primaries, color='magenta',
            size=25))

        equilibrium=[[L1(mu,TOL)[0],L1(mu,TOL)[1]],[L2(mu,
            TOL)[0],L2(mu,TOL)[1]],[L3(mu,TOL)[0],L3(mu,TOL)
            [1]],[L4(mu)[0],L4(mu)[1]],[L5(mu)[0],L5(mu)[1]]]
        plots.append(list_plot(equilibrium, color='cyan',
            size=25))



        plot_total=sum(plots)
        #plot_total.set_axes_range(1, 1.6, -0.2, 0.2)
        plot_total.axes_labels(['x','y'])
        plot_total.show(gridlines=True)
        plot_total.save(f'A11-W(L3)_{iregion}.png',dpi=300)


#OPTION 6: Plot the unstable curve of just one periodic
    point in the LPO
if option==6:
    C=3.189
    x=1.06102010630636
    T=periodic_orbit(x,C,mu,tf,timedirection,TOL,2)[0]
    x0=IC_xaxis(x,C,mu)
    evals,evecs=charmult(x0,T,mu,TOL)
    print(evals[0].real)
    print(evecs[0])

    type=0 #Unstable
    i=3 #L3
    iregion=+1 #Right branch


    plots=[]

    points=unstable_manifold_point(iregion,x0,T,100,2,mu,TOL
        )
    for p in points:
        x_list_u=p
        plots.append(list_plot(list(zip(x_list_u[0],x_list_u
            [1])),color='fuchsia',plotjoined=True))

    points=invariant_manifold_Li(type,i,iregion,100,2,mu,TOL
        )
    for p in points:
        x_list_u=p
```

```
        plots.append(list_plot(list(zip(x_list_u[0],x_list_u
            [1])),color='green',plotjoined=True))


    primaries=[P1(mu),P2(mu)]
    plots.append(list_plot(primaries, color='magenta', size
        =25))

    equilibrium=[[L1(mu,TOL)[0],L1(mu,TOL)[1]],[L2(mu,TOL)
        [0],L2(mu,TOL)[1]],[L3(mu,TOL)[0],L3(mu,TOL)[1]],[L4(
        mu)[0],L4(mu)[1]],[L5(mu)[0],L5(mu)[1]]]
    plots.append(list_plot(equilibrium, color='cyan', size
        =25))

    plot_total=sum(plots)
    plot_total.set_axes_range(1, 1.6, -0.2, 0.2)
    plot_total.axes_labels(['x','y'])
    plot_total.show(gridlines=True)
    plot_total.save(f'A11-unstablecurve.png',dpi=300)


#OPTION 7: Plot the unstable manifold of the LPO
if option==7:
    C=3.189
    x=1.06102010630636

    type=0 #Unstable
    i=3 #L3



    #PLOT 1: Positive branch, 2 crossings

    if plot==1:
        iregion=+1 #Right branch
        ncrossing=2

        norbits=50
        points_total=unstable_manifold_LPO(iregion,x,C,200,
            ncrossing,norbits,mu,TOL)
        plots=[]

        for points in points_total:
            for p in points:
                x_list_u=p
                plots.append(list_plot(list(zip(x_list_u[0],
                    x_list_u[1])),color='fuchsia',plotjoined=
                    True,thickness=0.15))
```

```
    x_list_u=periodic_orbit(x,C,mu,tf,timedirection,TOL
        ,2)[1]
    plots.append(list_plot(list(zip(x_list_u[0],x_list_u
        [1])),color='black',plotjoined=True))

    points=invariant_manifold_Li(type,i,iregion,100,2,mu
        ,TOL)
    for p in points:
        x_list_u=p
        plots.append(list_plot(list(zip(x_list_u[0],
            x_list_u[1])),color='green',plotjoined=True))


    primaries=[P1(mu),P2(mu)]
    plots.append(list_plot(primaries, color='magenta',
        size=25))

    equilibrium=[[L1(mu,TOL)[0],L1(mu,TOL)[1]],[L2(mu,
        TOL)[0],L2(mu,TOL)[1]],[L3(mu,TOL)[0],L3(mu,TOL)
        [1]],[L4(mu)[0],L4(mu)[1]],[L5(mu)[0],L5(mu)[1]]]
    plots.append(list_plot(equilibrium, color='cyan',
        size=25))

    plot_total=sum(plots)
    plot_total.axes_labels(['x','y'])
    plot_total.show(gridlines=True)
    plot_total.save(f'A11-unstable-manifold-positive.png
        ',dpi=300)

    plot_total.set_axes_range(1, 1.15, -1, 0.05)
    plot_total.save(f'A11-unstable-manifold-positive-
        zoom.png',dpi=300)



#PLOT 2: Negative branch, 2 crossings

if plot==2:
    iregion=-1 #Left branch
    ncrossing=2

    norbits=50
    points_total=unstable_manifold_LPO(iregion,x,C,200,
        ncrossing,norbits,mu,TOL)
    plots=[]

    for points in points_total:
        for p in points:
            x_list_u=p
```

```
            plots.append(list_plot(list(zip(x_list_u[0],
                x_list_u[1])),color='fuchsia',plotjoined=
                True,thickness=0.15))


    x_list_u=periodic_orbit(x,C,mu,tf,timedirection,TOL
        ,2)[1]
    plots.append(list_plot(list(zip(x_list_u[0],x_list_u
        [1])),color='black',plotjoined=True))

    points=invariant_manifold_Li(type,i,iregion,100,2,mu
        ,TOL)
    for p in points:
        x_list_u=p
        plots.append(list_plot(list(zip(x_list_u[0],
            x_list_u[1])),color='green',plotjoined=True))


    primaries=[P1(mu),P2(mu)]
    plots.append(list_plot(primaries, color='magenta',
        size=25))

    equilibrium=[[L1(mu,TOL)[0],L1(mu,TOL)[1]],[L2(mu,
        TOL)[0],L2(mu,TOL)[1]],[L3(mu,TOL)[0],L3(mu,TOL)
        [1]],[L4(mu)[0],L4(mu)[1]],[L5(mu)[0],L5(mu)[1]]]
    plots.append(list_plot(equilibrium, color='cyan',
        size=25))

    plot_total=sum(plots)
    plot_total.axes_labels(['x','y'])
    plot_total.show(gridlines=True)
    plot_total.save(f'A11-unstable-manifold-negative.png
        ',dpi=300)




#PLOT 3: Positive branch, 1 crossing. (y,x') at crossing

if plot==3:
    iregion=+1 #Right branch
    ncrossing=1

    norbits=150
    points_total=unstable_manifold_LPO(iregion,x,C,200,
        ncrossing,norbits,mu,TOL)

    intersection_points=[]

    for points in points_total:
```

```
            for points1 in points:
                point=points1.T[-1]
                print(point)
                intersection_points.append([point[1],point
                    [2]])


        plot_total=list_plot(intersection_points,color='blue
            ',plotjoined=False)
        plot_total.axes_labels(['y', "x'"])
        plot_total.show(gridlines=True)
        plot_total.save(f'A11-unstable-manifold-
            crossingprojection-positive.png',dpi=300)



    #PLOT 4: Negative branch, 1 crossing. (y,x') at crossing

    if plot==4:
        iregion=-1 #Left branch
        ncrossing=1

        norbits=150
        points_total=unstable_manifold_LPO(iregion,x,C,200,
            ncrossing,norbits,mu,TOL)

        intersection_points=[]

        for points in points_total:
            for points1 in points:
                point=points1.T[-1]
                print(point)
                intersection_points.append([point[1],point
                    [2]])


        plot_total=list_plot(intersection_points,color='blue
            ',plotjoined=False)
        plot_total.axes_labels(['y', "x'"])
        plot_total.show(gridlines=True)
        plot_total.save(f'A11-unstable-manifold-
            crossingprojection-negative.png',dpi=300)
```